Perfecto. Aquí vamos a hacer algo serio: un **documento maestro SOLO del dashboard**, pensado para que otro agente (OpenClaw o cualquier builder) lo construya **sin inventar nada**.

Esto no es inspiración, es **especificación técnica + UX + animaciones + integración real**.

🧠 DOCUMENTO MAESTRO

Dashboard de Prospección Aguas Arauka (GSAP + Supabase + OpenClaw)

---

1. 🎯 OBJETIVO DEL DASHBOARD

Construir un dashboard tipo CRM que permita:

- Visualizar prospectos (empresas/eventos)
- Filtrar por ubicación (Caracas, La Guaira, Aragua, Carabobo)
- Ver mapa geográfico de oportunidades
- Registrar notas y seguimiento
- Activar acciones (llamar, contactar)
- Visualizar actividad del agente
- Interactuar con el agente (tipo chat)

👉 Este dashboard es el **centro de control del sistema autónomo**

---

2. 🧱 STACK TECNOLÓGICO

Frontend:

- Next.js (App Router)
- Tailwind CSS
- GSAP (animaciones avanzadas)
- React Query (data fetching)

Backend:

- Supabase (DB + auth)
- Edge Functions (opcional)

Integraciones:

- OpenClaw Agent API (local o endpoint)
- Telegram Bot (opcional bridge)

---

3. 🧭 ESTRUCTURA GENERAL DEL DASHBOARD

```
/app
  /dashboard
    /layout
    /components
    /modules
      - mapa
      - tabla_prospectos
      - detalle_empresa
      - actividad_agente
      - chat_agente
```

---

4. 🧩 MÓDULOS PRINCIPALES

4.1 📊 Vista Principal (Overview)

Componentes:

- KPIs:
  - Total prospectos
  - Nuevos hoy
  - Contactados
  - Pendientes
- Gráficas:
  - Prospectos por ciudad
  - Prospectos por tipo de evento

🎬 Animaciones GSAP

- Entrada inicial:
  - fade + slide staggered cards
- KPIs:
  - count-up animado
- Gráficas:
  - reveal progresivo

4.2 🗺️ Módulo de Mapa (CRÍTICO)

Funcionalidad:

- Mostrar prospectos geolocalizados
- Agrupar por clusters
- Click → abrir detalle

Librería:

- Mapbox GL o Leaflet

🎬 Animaciones GSAP en mapa

- Zoom inicial suave
- Aparición de markers con scale bounce
- Hover → expansión + glow
- Selección → transición a panel lateral

4.3 📋 Tabla de Prospectos

Columnas:

- Empresa
- Evento
- Ciudad
- Tipo
- Teléfono
- Estado
- Fecha

Funciones:

- Filtros dinámicos
- Búsqueda en tiempo real
- Ordenamiento
- Click → abrir detalle

🎬 Animaciones:

- Filtrado:
  - fade + reflow animado
- Hover fila:
  - highlight + scale leve
- Cambio de página:
  - slide horizontal

4.4 🧾 Panel Detalle Empresa

Datos:

- Información completa
- Historial de eventos
- Contacto
- Notas

Acciones:

- 📞 botón llamar
- ✉️ enviar mensaje
- 📝 agregar nota
- ✔️ cambiar estado

🎬 Animaciones:

- Slide-in lateral

- Tabs con transición fluida
- Input notas con focus glow

4.5 🤖 Actividad del Agente

Mostrar:

- Últimas búsquedas
- URLs visitadas
- Prospectos encontrados

UI tipo log:

[10:32] Buscando eventos en Caracas...
[10:33] 12 empresas encontradas
[10:34] Analizando Instagram...

Animaciones:

- Scroll automático
- Entrada tipo terminal (typing effect)

4.6 💬 Chat con el Agente (MUY IMPORTANTE)

Funcionalidad:

- Preguntar:
  - “¿Qué eventos hay en Valencia?”
  - “Dame prospectos nuevos”
- Ejecutar acciones:
  - “Busca eventos deportivos en Caracas”

UI:

- Input + historial
- Respuestas enriquecidas

🎬 Animaciones:

- typing animation
- aparición tipo burbuja
- loading con pulso

---

5. 🔌 INTEGRACIÓN CON OPENCLAW

Endpoint esperado:

POST /agent/run

Request:

```
{
  "input": "buscar eventos en caracas"
}
```

Response:

```
{
  "response": "Encontré 15 nuevos eventos...",
  "actions": [],
  "data": []
}
```

En frontend:

```
const sendToAgent = async (input) => {
  const res = await fetch("/api/agent", {
    method: "POST",
    body: JSON.stringify({ input })
  });
  return res.json();
};
```

---

6. 🗄️ INTEGRACIÓN CON SUPABASE

Tabla: prospectos

Campos clave:

- empresa
- evento
- ciudad
- lat
- lng
- telefono
- estado
- notas

Query ejemplo:

```
const { data } = await supabase
  .from("prospectos")
  .select("*")
  .eq("ciudad", "Caracas");
```

---

7. 🎨 SISTEMA DE DISEÑO

Estilo:

- Dark mode profesional
- Glassmorphism ligero
- Bordes redondeados (2xl)
- Sombras suaves

Colores:

- Fondo: #0B0F1A
- Primario: azul eléctrico
- Acento: verde (acciones)

Tipografía:

- Inter / Geist

---

8. 🎬 SISTEMA DE ANIMACIONES GSAP (PRO)

Configuración base:

```
import gsap from "gsap";
import { ScrollTrigger } from "gsap/ScrollTrigger";

gsap.registerPlugin(ScrollTrigger);
```

Ejemplo entrada dashboard:

```
gsap.from(".card", {
  opacity: 0,
  y: 40,
  stagger: 0.1,
  duration: 0.8,
  ease: "power3.out"
});
```

Hover avanzado:

```
gsap.to(element, {
  scale: 1.05,
  boxShadow: "0 0 20px rgba(0,255,255,0.3)"
});
```

Animación mapa markers:

```
gsap.from(marker, {
  scale: 0,
  duration: 0.5,
  ease: "back.out(1.7)"
});
```

---

9. 📱 RESPONSIVE

- Desktop: completo
- Tablet: sidebar colapsable
- Mobile:
    - mapa fullscreen
    - tabla simplificada

---

10. 🚨 REGLAS PARA EL AGENTE CONSTRUCTOR (CRÍTICO)

Esto evita que “alucine”:

❗ REGLAS

1. NO inventar endpoints
2. NO cambiar estructura DB
3. NO usar librerías no especificadas
4. GSAP obligatorio para animaciones
5. Mapbox o Leaflet obligatorio
6. Toda data viene de Supabase
7. El agente SOLO se conecta vía `/api/agent`

---

11. 🚀 FUTURAS MEJORAS

- scoring de leads
- predicción de eventos
- automatización WhatsApp
- alertas en tiempo real

---

12. 🧠 CONCLUSIÓN

Este dashboard no es solo visualización:

👉 es un **centro de operaciones comercial**

Integra:

- inteligencia artificial
- scraping
- CRM
- automatización

💬 SIGUIENTE PASO

Si quieres, puedo:

- darte el **código completo en Next.js listo para copiar**
- diseñarte el **UI exacto tipo Figma**
- o crear el **prompt perfecto para el agente que construya todo esto**

Solo dime:

👉 “quiero el código del dashboard”
o
👉 “quiero el prompt para el agente constructor” 🚀